

# Revision

## Lab 3

The code you've provided contains four examples of different signal manipulations using MATLAB. Each example demonstrates a particular concept in signal processing, such as adding signals, scaling, folding, and shifting. Below is a breakdown of the code with explanations and an organized structure.

---

### Example 1: Adding Signals

**Goal:** Create two signals (a sine and a cosine wave) and add them to form a new signal.

```
t = 0:0.01:0.1;    % Time vector from 0 to 0.1 with 0.01 interval
f = 25;            % Frequency of the signal
t1 = 2 * pi * f * t; % Time argument for sine and cosine waves

x1 = sin(t1);       % First signal: sine wave
subplot(3,1,1)      % Create a subplot for displaying the first signal
stem(t, x1)         % Plot the first signal
title("Signal One");

x2 = cos(t1);       % Second signal: cosine wave
subplot(3,1,2)      % Create a subplot for displaying the second signal
stem(t, x2);        % Plot the second signal
title("Signal Two");

x3 = x1 + x2;       % New signal: sum of the sine and cosine signals
subplot(3,1,3)      % Create a subplot for displaying the summed signal
```

```

stem(t+2, x3)      % Plot the new signal with a time shift of
2 units
title("New Signal");
xlabel('Time');     % Label for the x-axis
ylabel('Amplitude'); % Label for the y-axis

```

### Explanation:

- **Time vector:** `t` spans from 0 to 0.1 seconds with a step of 0.01 seconds.
- **Signal generation:** `x1` is a sine wave and `x2` is a cosine wave, both having the same frequency of 25 Hz.
- **Addition of signals:** `x3` is the sum of `x1` and `x2`.
- **Subplots:** Each signal (`x1`, `x2`, and `x3`) is plotted in a separate subplot.

## Example 2: Scaling a Signal

**Goal:** Scale the amplitude of a signal.

```

t = 0:0.01:0.1;    % Time vector from 0 to 0.1 with 0.01 interval
f = 25;             % Frequency of the signal
t1 = 2 * pi * f * t; % Time argument for sine wave

x1 = sin(t1);       % Original signal: sine wave
subplot(2,1,1)      % Create a subplot for displaying the original signal
stem(t, x1)         % Plot the original signal
title("Original Signal");
xlabel('Time');
ylabel('Amplitude');
grid on;

x2 = 4 * x1;        % Scaled signal: amplitude is multiplied by 4
subplot(2,1,2)      % Create a subplot for displaying the scaled signal

```

```

ed signal
stem(t, x2);      % Plot the scaled signal
title("Scaled Signal");
xlabel('Time');
ylabel('Amplitude');
grid on;

```

#### Explanation:

- **Signal scaling:** The amplitude of the sine wave ( `x1` ) is multiplied by 4 to produce a new signal `x2` .
- **Subplots:** The original and scaled signals are displayed in two subplots.

### Example 3: Folding a Signal

**Goal:** Fold (reverse) the original signal.

```

t = -1:0.1:1;      % Time vector from -1 to 1 with 0.1 interval
x1 = sin(t + 1);   % Original signal: sine wave shifted by 1

subplot(2,1,1)     % Create a subplot for displaying the original signal
stem(t, x1)        % Plot the original signal
title("Original Signal");
xlabel('Time');
ylabel('Amplitude');
grid on;

x2 = fliplr(x1);    % Folded signal: reverse the signal using fliplr function
subplot(2,1,2)     % Create a subplot for displaying the folded signal
stem(t, x2);       % Plot the folded signal
title("Folded Signal");
xlabel('Time');

```

```
ylabel('Amplitude');  
grid on;
```

#### Explanation:

- **Signal folding:** The function `fliplr(x1)` reverses the signal (`x1`) in time, producing the folded signal `x2`.
- **Subplots:** Both the original and folded signals are displayed in separate subplots.

### Example 4: Shifting a Signal (Delay and Advance)

**Goal:** Demonstrate delay and advance shifting of a signal.

```
t = -1:0.1:1;          % Time vector from -1 to 1 with 0.1 interval  
x1 = sin(t * pi);      % Original signal: sine wave with pi factor  
  
subplot(3,1,1)         % Create a subplot for displaying the original signal  
stem(t, x1)            % Plot the original signal  
title("Original Signal");  
xlabel('Time');  
ylabel('Amplitude');  
grid on;  
axis([-4, 4, -1, 1]); % Set axis limits  
  
t1 = t + 2;            % Time vector for delay shifting  
subplot(3,1,2)         % Create a subplot for displaying the delayed signal  
stem(t1, x1);          % Plot the delayed signal (shifted by 2 units)  
title("Delay Shifting");  
xlabel('Time');  
ylabel('Amplitude');  
grid on;
```

```
axis([-4, 4, -1, 1]);

subplot(3,1,3)      % Create a subplot for displaying the advanced signal
stem(t - 2, x1);    % Plot the advanced signal (shifted by -2 units)
title("Advance Shifting");
xlabel('Time');
ylabel('Amplitude');
grid on;
axis([-4, 4, -1, 1]);
```

### Explanation:

- **Signal shifting:** The time vector is modified to create two new signals:
  - **Delay:** The signal is shifted by +2 units in time (  $t_1 = t + 2$  ).
  - **Advance:** The signal is shifted by -2 units in time (  $t - 2$  ).
- **Subplots:** The original, delayed, and advanced signals are displayed in separate subplots.

### Summary

- **Example 1:** Demonstrates adding two signals (sine and cosine) to create a new signal.
- **Example 2:** Shows how to scale a signal by changing its amplitude.
- **Example 3:** Explains folding (reversing) a signal.
- **Example 4:** Illustrates how to delay or advance a signal by shifting its time axis.

Each example uses MATLAB's `stem()` function to plot discrete signals and utilizes `subplot()` for organizing multiple plots in a single figure.

## lab 4

This code demonstrates the concept of **sampling a continuous sinusoidal signal** at different frequencies to show the effects of **undersampling**, **perfect sampling**, and **oversampling**. The

code also uses MATLAB's plotting functions to visually compare the signals. Below is an organized explanation of each part.

---

## Overview of Parameters:

- **f** (signal frequency): The frequency of the continuous sinusoidal signal, set to 1000 Hz.
  - **fs** (sampling frequency): Sampling frequency used for generating the signal at different rates (varied across examples).
  - **nCyl** (number of cycles): The number of cycles of the sinusoidal signal to be plotted.
  - **t** (time vector): Represents the time axis for the signal.
- 

## Code Breakdown

### Part 1: Continuous Sinusoidal Signal (Over-Sampling)

```
f = 1000; % Signal frequency
fs = 5e5; % High sampling frequency (500kHz)
nCyl = 5; % Number of cycles
t = 0:1/fs:nCyl*1/f; % Time vector for high sampling rate
y = sin(2*pi*f*t); % Continuous sinusoidal signal

subplot(4,1,1);
plot(t, y, 'b'); % Plot continuous sinusoidal signal
title('Continuous sinusoidal signal of frequency=1000Hz');
xlabel('Time(sec)');
ylabel('Amplitude Y(t)');
axis([0 nCyl/f -1.1 1.1]); % Time range adjusted for the number of cycles
```

### Explanation:

- A continuous sinusoidal signal is created using `sin(2*pi*f*t)`, where **f** is the signal frequency (1000 Hz).
- The signal is plotted using `plot(t, y, 'b')` in blue, showing the continuous signal.

- The axis is set to display the first 5 cycles of the signal.

## Part 2: Undersampling ( $f_s = 1500$ Hz)

```
fs1 = 1500; % Undersampling frequency (less than Nyquist frequency)
t1 = 0:1/fs1:nCyl*1/f; % Time vector for undersampling
y1 = sin(2*pi*f*t1); % Sampled signal at fs1

subplot(4,1,2);
plot(t, y, 'r'); % Plot continuous signal
hold on;
plot(t1, y1, ':'); % Plot undersampled signal
hold on;
stem(t1, y1, 'y', 'filled'); % Plot sampled points using stem
title('Continuous sinusoidal signal sampled at 1500Hz < Nyquist (2000Hz)');
xlabel('Time(sec)');
ylabel('Amplitude Y(t)');
axis([0 nCyl/f -1.1 1.1]);
```

### Explanation:

- **Undersampling** occurs when the sampling frequency  $f_{s1}$  is below the Nyquist frequency (2000 Hz for a 1000 Hz signal). In this case,  $f_{s1} = 1500$  Hz.
- The signal is sampled at 1500 Hz and plotted as a discrete set of points ( `stem(t1, y1, 'y', 'filled')` ).
- This produces an aliased version of the signal, where the sampled points no longer accurately represent the original continuous signal.

## Part 3: Oversampling ( $f_s = 4000$ Hz)

```
fs2 = 4000; % Oversampling frequency (greater than Nyquist frequency)
t2 = 0:1/fs2:nCyl*1/f; % Time vector for oversampling
```

```

y2 = sin(2*pi*f*t2); % Sampled signal at fs2

subplot(4,1,3);
plot(t, y, 'r'); % Plot continuous signal
hold on;
plot(t2, y2, ':'); % Plot oversampled signal
hold on;
stem(t2, y2, 'g', 'filled'); % Plot sampled points using stem
title('Continuous sinusoidal signal sampled at 4000Hz > Nyquist');
xlabel('Time(sec)');
ylabel('Amplitude Y(t)');
axis([0 nCyl/f -1.1 1.1]);

```

#### Explanation:

- **Oversampling** occurs when the sampling frequency `fs2` is higher than the Nyquist frequency (2000 Hz). Here, `fs2 = 4000` Hz.
- The signal is sampled at 4000 Hz and plotted using both `plot` (to show the continuous sinusoidal signal) and `stem` (to show the discrete samples).
- The oversampling provides a much smoother representation of the signal.

#### Part 4: Perfect Sampling ( $f_s = 2000$ Hz)

```

fs3 = 2000; % Perfect sampling frequency (exactly at Nyquist rate)
t3 = 0:1/fs3:nCyl*1/f; % Time vector for perfect sampling
y3 = sin(2*pi*f*t3); % Sampled signal at fs3

subplot(4,1,4);
plot(t, y, 'r'); % Plot continuous signal
hold on;
plot(t3, y3, ':'); % Plot perfectly sampled signal
hold on;
stem(t3, y3, 'r', 'filled'); % Plot sampled points using stem

```



```
title('Continuous sinusoidal signal sampled at 2000Hz = Nyquist');
xlabel('Time(sec)');
ylabel('Amplitude Y(t)');
axis([0 nCyl/f -1.1 1.1]);
```

### Explanation:

- **Perfect sampling** occurs when the sampling frequency `fs3` is exactly at the Nyquist frequency (2000 Hz).
- The signal is sampled at 2000 Hz and the discrete points are plotted, which perfectly match the continuous signal without any aliasing.

---

### Final Output:

- **Figures:** Four subplots are generated, each showing a different sampling rate:
  1. **Continuous signal** at 1000 Hz (oversampled at 500,000 Hz).
  2. **Undersampled signal** at 1500 Hz (below Nyquist).
  3. **Oversampled signal** at 4000 Hz (above Nyquist).
  4. **Perfectly sampled signal** at 2000 Hz (exactly at Nyquist).
- **PNG Export:** The figure is saved as a PNG image file called `'sampling.png'` using `print('sampling', '-dpng')`.

### Summary of Sampling Rates:

- **Undersampling** (1500 Hz): Sampling below the Nyquist frequency (2000 Hz), leading to aliasing and loss of information.
  - **Perfect sampling** (2000 Hz): Sampling exactly at the Nyquist frequency, capturing the signal without distortion.
  - **Oversampling** (4000 Hz): Sampling above the Nyquist frequency, providing a smooth representation but not necessary for accurate reconstruction.
-

# Lab 5 , 6

Here's a structured explanation of the MATLAB code in a format suited for **Notion**, focusing on key signal processing concepts like **decimation**, **interpolation**, and **sampling**:

---

## Signal Processing Concepts in MATLAB

### 1. Introduction

The following MATLAB code demonstrates key signal processing concepts including **decimation**, **interpolation**, and **sampling**. These concepts are essential for understanding how signals can be manipulated, reduced in resolution (decimation), and later reconstructed (interpolation).

---

### 2. Example 1: Signal Decimation and Interpolation

#### Concepts Covered:

- **Decimation:** Reducing the sampling rate of a signal.
- **Interpolation:** Increasing the sampling rate to reconstruct the signal.

#### Code:

```
t = 0:0.01:1; % Time vector from 0 to 1 with a step of 0.01
x = sin(2*pi*5*t); % Original sinusoidal signal with frequency 5Hz
y = decimate(x,4); % Decimate the signal by a factor of 4
y1 = interp(y,4); % Interpolate the decimated signal by a factor of 4

% Plotting the original, decimated, and interpolated signals
subplot(3,1,1);
stem(x, 'filled', 'MarkerSize', 3); % Original signal plot
```

```

grid on;
xlabel('Sample Number');
ylabel('Original');

subplot(3,1,2);
stem(y, 'filled', 'MarkerSize', 3); % Decimated signal plot
grid on;
xlabel('Sample Number');
ylabel('Decimated');

subplot(3,1,3);
stem(y1, 'filled', 'MarkerSize', 3); % Interpolated signal plot
grid on;
xlabel('Sample Number');
ylabel('Interpolation');

```

#### Explanation:

- The **original signal** ( x ) is a sinusoidal wave sampled at 100 Hz.
- **Decimation:** Reduces the sampling rate by a factor of 4 using the decimate function, resulting in fewer samples of the original signal.
- **Interpolation:** The interp function is used to reconstruct the signal by increasing its sampling rate by a factor of 4.
- The signals are visualized in three subplots:
  1. **Original Signal** (in blue).
  2. **Decimated Signal** (in red).
  3. **Interpolated Signal** (in green).

### 3. Example 2: Shortened Signal View

#### Concepts Covered:

- **Zooming In:** Focus on a smaller portion of the signal.

## Code:

```
subplot(3,1,1);
stem(x(1:25), 'filled', 'MarkerSize', 3); % Show only the first 25 samples of the original signal
grid on;
xlabel('Sample Number');
ylabel('Original');

subplot(3,1,2);
stem(y, 'filled', 'MarkerSize', 3); % Decimated signal plot
grid on;
xlabel('Sample Number');
ylabel('Decimated');

subplot(3,1,3);
stem(y1(1:25), 'filled', 'MarkerSize', 3); % Show first 25 samples of the interpolated signal
grid on;
xlabel('Sample Number');
ylabel('Interpolation');
```

## Explanation:

- This example restricts the plot to the **first 25 samples** of each signal for a more detailed look at the initial samples.
- It demonstrates how to zoom into a portion of a signal for closer inspection.

---

## 4. Example 3: Decimation of Composite Signal

### Concepts Covered:

- **Composite Signal:** A signal made up of multiple sinusoidal components.
- **Sampling and Decimation:** The process of sampling at a certain rate and reducing the sampling rate.

## Code:

```
t = 0:1/4e3:1; % Time vector with sa
mpling rate 4000Hz
x = sin(2*pi*30*t) + sin(2*pi*60*t); % Composite signal w
ith frequencies 30Hz and 60Hz
y = decimate(x,4); % Decimate the signa
l by a factor of 4

% Plotting the original and decimated signals
subplot(3,1,1);
stem(x(1:121), 'filled', 'MarkerSize', 3); % Original signal
plot
grid on;
xlabel('Sample Number');
ylabel('Original');

subplot(3,1,2);
stem(y(1:31), 'filled', 'MarkerSize', 3); % Decimated signa
l plot
grid on;
xlabel('Sample Number');
ylabel('Decimated');

yd = interp(y,4); % Interpolating the
decimated signal
subplot(3,1,3);
stem(yd(1:121), 'filled', 'MarkerSize', 3); % Interpolated si
gnal plot
grid on;
xlabel('Sample Number');
ylabel('Interpolation');
```

## Explanation:

- A **composite signal** is created by adding two sinusoids with different frequencies (30 Hz and 60 Hz).
  - The signal is then **decimated** by a factor of 4, reducing the sampling rate.
  - The **interpolated signal** is obtained by increasing the sampling rate back.
  - The signals are plotted in three subplots:
    1. **Original Signal** (in blue).
    2. **Decimated Signal** (in red).
    3. **Interpolated Signal** (in green).
- 

## 5. Example 4: Sampling and Reconstruction

### Concepts Covered:

- **Sampling Rate:** The rate at which a continuous signal is sampled to produce discrete samples.
- **Reconstruction:** The process of recovering the original signal from its discrete samples.

### Code:

```
f = 2000; % Signal frequenc
y 2000Hz
t = 0: 0.00001 : 3*1/f; % Continuous time
vector
y = sin(2*pi*f*t); % Continuous sign
al

subplot(2,1,1)
plot(t, y); % Plot the contin
uous signal

fs = 50000; % Sampling rate 5
0000Hz
ts = 0: 1/fs : 3*1/f; % Discrete time v
ector for samples
```

```

ys = sin(2*pi*f*ts);                                % Sampled signal

subplot(2,1,2)
hold on
plot(t, y, 'b');                                     % Plot continuous
signal (in blue)
stem(ts, ys, 'r');                                   % Plot sampled si
gnal (in red)
hold off

```

### Explanation:

- A **continuous sinusoidal signal** is created with a frequency of 2000 Hz.
- The signal is then **sampled** at a high rate of 50,000 Hz.
- The first subplot shows the **continuous signal**.
- The second subplot shows both the **continuous signal** (in blue) and the **sampled signal** (in red) using `stem`.

## 6. Conclusion

These examples showcase various signal processing techniques using **MATLAB**, including:

- **Decimation** to reduce the sampling rate of a signal.
- **Interpolation** to reconstruct signals after decimation.
- **Sampling** a continuous signal and visualizing both the continuous and sampled versions.

This approach to signal processing is crucial for applications such as data compression, signal reconstruction, and efficient transmission of signals.

This content is now structured in a clean and easy-to-follow manner for **Notion**, with each concept clearly explained and the corresponding code and output behavior highlighted.

## Lab 7

Here's how you can structure the **Linear Convolution** example in **Notion**, including the explanation, MATLAB code, and outputs:

---

# Linear Convolution in MATLAB

## 1. Introduction

In signal processing, **convolution** is an operation that combines two sequences to produce a third. The result of the convolution is often used in systems analysis, filtering, and signal processing tasks. The example below demonstrates **linear convolution** between two sequences, `x` and `h`.

---

## 2. MATLAB Code for Linear Convolution

```
clear all;
close all;
clc;

% Linear Convolution
x = [2, 1, 3, 5]; % Input sequence
h = [2, 4, 1];    % Impulse response

% Convolution of x and h
Y = conv(x, h);

% Plotting the sequences and their convolution result
subplot(3, 1, 1);
stem(x, 'filled');
title('Sequence x : [2, 1, 3, 5]');
xlabel('n i.e. sample number');
ylabel('X(n)');

subplot(3, 1, 2);
stem(h, 'filled');
title('Sequence h : [2, 4, 1]');
xlabel('n i.e. sample number');
ylabel('Y(n)');
```



```
subplot(3, 1, 3);
stem(Y, 'filled');
xlabel('n i.e. sample number');
title({'Linear Convolution of Sequence x and h', num2str
(Y)});
ylabel('Convolved Signal');
```

### 3. Explanation

- **Input Sequences:**

- `x = [2, 1, 3, 5]` : This is the first sequence (`x`).
- `h = [2, 4, 1]` : This is the second sequence (`h`), which is usually considered as the **impulse response** of the system.

- **Convolution Operation:**

The

`conv(x, h)` function in MATLAB performs the **linear convolution** of `x` and `h`, resulting in sequence `Y`.

### 4. Output Plots

#### Plot 1: Sequence `x`

- This plot shows the input sequence `x = [2, 1, 3, 5]`.

#### Plot 2: Sequence `h`

- This plot shows the impulse response sequence `h = [2, 4, 1]`.

#### Plot 3: Convolved Signal

- This plot shows the result of the linear convolution of `x` and `h`, resulting in sequence `Y`. The MATLAB code also displays the resulting sequence values in the title of the plot.
  - The resulting **convolution sequence** `Y` is calculated as:
    - `Y = conv(x, h) = [4, 12, 11, 17, 13, 5]`.

## 5. Conclusion

The linear convolution of sequences `x` and `h` gives the result `Y = [4, 12, 11, 17, 13, 5]`. The convolution process effectively combines the two sequences, and it can be visualized step by step using MATLAB's `conv()` function. This operation is fundamental in signal processing for tasks like filtering and systems analysis.

---

This content is structured to clearly present the **linear convolution process** using MATLAB, with corresponding code, explanation, and visualization.

---

## lab 8

Here is an organized explanation of the provided MATLAB code for the second-order system, pole-zero analysis, and linear convolution:

---

### 1. Difference Equation of a Second-Order System

#### 1.1. System Definition

The difference equation for the second-order system is defined as:

$$y(n) = x(n) + 0.5 \cdot x(n-1) + 0.85 \cdot x(n-2) + y(n-1) + y(n-2)$$

Where:

- `x(n)` is the input signal.
- `y(n)` is the output signal.

#### 1.2. Impulse Response Calculation

The impulse response `h(n)` of this system is computed using the `impz` function in MATLAB, which is used to calculate the impulse response for a system defined by the numerator `b` and denominator `a` coefficients.

```
b = [1 0.5 0.85]; % Numerator coefficients (x terms)
a = [1 -1 -1];    % Denominator coefficients (y terms)
```

```
N = 4; % Number of points for impulse response
[h, t] = impz(b, a, N); % Impulse response of the system
```

### 1.3. Plot of Impulse Response

Two plots are generated to visualize the impulse response of the system:

1. **Continuous plot** using `plot(t, h)`.
2. **Stem plot** using `stem(t, h)`.

These plots show the impulse response of the system over time.

```
subplot(2, 1, 1);
plot(t, h); % Continuous plot
title('Plot of Impulse Response');
ylabel('Amplitude');
xlabel('Time Index (N)');

subplot(2, 1, 2);
stem(t, h); % Stem plot
title('Plot of Impulse Response');
ylabel('Amplitude');
xlabel('Time Index (N)');
```

### 1.4. Output of Impulse Response

The impulse response is displayed in the command window using `disp(h)`.

## 2. Pole-Zero Analysis

### 2.1. System 1:

For the transfer function:

$$H(z) = \frac{1 + 0z}{3 - 3z + z^2}$$

The poles, zeros, and gain are found using the `tf2zpk` function.

```

b = [1 0];           % Numerator coefficients (zeros)
a = [3 -3 1];        % Denominator coefficients (poles)
[z, p, k] = tf2zpk(b, a); % Find poles, zeros, and gain
zplane(z, p);        % Plot pole-zero plot

```

## 2.2. System 2:

For the transfer function:

$$H(z) = \frac{1 + 2z^{-1} - 1.2z^{-2}}{1 - 1.2z^{-1} + 2z^{-2}}$$

The poles, zeros, and gain are again calculated.

```

b = [1 2];           % Numerator coefficients (zeros)
a = [1 -1.2 2];       % Denominator coefficients (poles)
[z, p, k] = tf2zpk(b, a); % Find poles, zeros, and gain
zplane(z, p);        % Plot pole-zero plot

```

## 3. Linear Convolution

### 3.1. System Definition

The input sequence `x` and the impulse response `h` are defined as follows:

```

x = [2, 1, 3, 5]; % Input sequence
h = [2, 4, 1];    % Impulse response

```

### 3.2. Convolution Calculation

The convolution of `x` and `h` is calculated using the `conv` function in MATLAB.

```

Y = conv(x, h); % Linear convolution

```

### 3.3. Plot of Convolution

Three plots are created to show:

1. **Original sequence**  $x$  using a stem plot.
2. **Impulse response**  $h$  using a stem plot.
3. **Convolved signal**  $Y$  using a stem plot and displaying the convolution result in the title.

```
subplot(3, 1, 1);
stem(x, 'filled');
title('Sequence x: [2, 1, 3, 5]');
xlabel('n (Sample Number)');
ylabel('X(n)');

subplot(3, 1, 2);
stem(h, 'filled');
title('Sequence h: [2, 4, 1]');
xlabel('n (Sample Number)');
ylabel('Y(n)');

subplot(3, 1, 3);
stem(Y, 'filled');
xlabel('n (Sample Number)');
title({'Linear Convolution of Sequence x and h', num2str
(Y)});
ylabel('Convolved Signal');
```

---

## 4. Conclusion

The script demonstrates:

1. **Impulse Response Calculation** for a second-order system.
2. **Pole-Zero Analysis** of different transfer functions.
3. **Linear Convolution** of two sequences to show the resulting output signal.

The impulse response and convolution operations help understand how a system responds to an impulse and how two signals combine in a linear system.

---

# Lab 9

The MATLAB code provided performs the Discrete Fourier Transform (DFT) of a sequence, calculates and plots its magnitude and phase response, and demonstrates the use of the Fast Fourier Transform (FFT) function along with its inverse. Here's a breakdown of the steps in your code:

## 1. Discrete Fourier Transform (DFT) Calculation

- **Input Sequence:** The sequence `xn` is given as `[2, 3, -1, 4]`.
- **DFT Calculation:** The DFT is computed manually using the formula:

$$X(k) = \sum_{n=0}^{N-1} x(n) \cdot \exp(-j \cdot 2\pi k n / N)$$
$$X(k) = \sum_{n=0}^{N-1} x(n) \cdot \exp\left(-j \cdot \frac{2\pi k n}{N}\right)$$

Where:

- `k` is the frequency index.
- `n` is the time index.
- `xn` is the input sequence.

```
xn = [2 3 -1 4]; % Input sequence
ln = length(xn); % Length of the sequence
dft = zeros(1, ln); % Initialize the DFT array

% DFT Calculation
for k = 0:ln-1
    for n = 0:ln-1
        dft(k+1) = dft(k+1) + (xn(n+1) * exp(-j*2*pi*k*n/ln));
    end
end
```

## 2. Plotting the Input Sequence and DFT Results

- **Plot 1:** The input sequence is plotted using `stem(t, xn)`.

```
t = 0:ln-1; % Time index
subplot(221);
stem(t, xn); % Plot the input sequence
ylabel('Amplitude');
xlabel('Time Index');
title('Input Sequence');
```

- **Plot 2:** The magnitude of the DFT is computed using `abs(dft)` and plotted.

```
magnitude = abs(dft); % Magnitude of DFT
subplot(222);
stem(t, magnitude); % Plot the magnitude response
ylabel('Amplitude');
xlabel('K');
title('Magnitude Response');
```

- **Plot 3:** The phase of the DFT is computed using `angle(dft)` and plotted.

```
phase = angle(dft); % Phase of DFT
subplot(223);
stem(t, phase); % Plot the phase response
ylabel('Phase');
xlabel('K');
title('Phase Response');
```

### 3. Fast Fourier Transform (FFT)

- The same sequence `x = [1, 2, 3, 4]` is used to demonstrate the FFT function in MATLAB.
- **FFT Calculation:** The FFT of the sequence is computed using `fft(x)`.

```
x = [1, 2, 3, 4]; % Another input sequence
Y = fft(x); % Fast Fourier Transform
disp('DFT Magnitude:');
disp(abs(Y)); % Display the magnitude of the FFT
```

```
disp('DFT Phase:');  
disp(unwrap(angle(Y))); % Display the phase (unwrap to avoid  
discontinuities)
```

- **Plot 1 (Time Domain):** The input sequence is plotted in the time domain using `stem(x)`.

```
subplot(2,2,1);  
stem(x); % Plot the time domain signal  
title('Time Domain');  
xlabel('Sample Index');  
ylabel('Amplitude');
```

- **Plot 2 (Frequency Domain):** The magnitude of the FFT is plotted in the frequency domain.

```
subplot(2,2,2);  
stem(abs(Y)); % Plot the magnitude of FFT  
title('Frequency Domain');  
xlabel('Frequency Index');  
ylabel('Magnitude');
```

- **Plot 3 (Inverse FFT):** The inverse FFT of `Y` is plotted to recover the original sequence.

```
subplot(2,2,3);  
stem(ifft(Y)); % Inverse FFT to recover the time-domain signal
```

## Summary of Key Points:

1. **Manual DFT Computation:** The code manually computes the DFT of a sequence using the definition of the Discrete Fourier Transform.
2. **FFT Function:** MATLAB's `fft()` function is used for efficient computation of the Fourier transform, and `ifft()` computes the inverse FFT.
3. **Magnitude and Phase:** The magnitude (`abs()`) and phase (`angle()`) of the DFT/FFT are calculated and displayed.



4. **Plots:** The sequence, its magnitude, and phase are visualized in different subplots.

This code gives a detailed view of both the manual DFT computation and the built-in FFT function in MATLAB, along with plotting for analysis in both the time and frequency domains.

---

## Lab 10

This MATLAB code creates a low-pass FIR filter using the `fir1` function with a Hamming window and plots its frequency response using `freqz`. Below is a detailed explanation of the code:

### Code Explanation:

#### 1. Define the Parameters:

- `N = 50;` : This sets the order of the FIR filter. The order of the filter determines the number of taps (or coefficients) in the filter. In this case, it is 50.
- `Fs = 2000;` : This is the sampling frequency in Hz (samples per second).
- `fcut = 600;` : This is the cutoff frequency for the low-pass filter in Hz.

#### 2. Normalize the Cutoff Frequency:

The cutoff frequency needs to be normalized to the Nyquist frequency (half of the sampling frequency):

```
Fc = fcut / (Fs / 2);
```

- `Fs / 2` is the Nyquist frequency, and `fcut` is divided by the Nyquist frequency to get a normalized cutoff frequency (`Fc`), which is required by `fir1`.

#### 3. Design the FIR Filter:

The filter is designed using the

`fir1` function:

```
b = fir1(N, Fc, 'low', hamming(N+1));
```

- `fir1(N, Fc, 'low', ...)` : This creates an FIR low-pass filter of order `N` with the normalized cutoff frequency `Fc`.

- `hamming(N+1)` : This specifies that a Hamming window of length `N+1` is applied to the filter design. The Hamming window is used to reduce side lobes in the frequency response and improve the filter's performance.

#### 4. Plot the Frequency Response:

The frequency response of the filter is plotted using

`freqz` :

```
freqz(b, 1);
title('Lowpass Filter using fir1 with Hamming Window');
```

- `freqz(b, 1)` : This function plots the frequency response of the FIR filter defined by the coefficients `b`. The second argument is `1` because the filter has a denominator of 1 (it is an FIR filter).
- `title('Lowpass Filter using fir1 with Hamming Window')` : This adds a title to the plot.

### Summary:

- The code designs a low-pass FIR filter with a cutoff frequency of 600 Hz and a sampling frequency of 2000 Hz using a Hamming window. It then plots the frequency response of the filter.

### Key Parameters:

- **Filter Order ( `N` )**: 50
- **Sampling Frequency ( `Fs` )**: 2000 Hz
- **Cutoff Frequency ( `fcut` )**: 600 Hz
- **Normalized Cutoff Frequency ( `Fc` )**: `600 / 1000 = 0.6`

The `freqz` plot will show the magnitude and phase response of the filter, indicating how the filter attenuates frequencies above 600 Hz and passes frequencies below this cutoff.